

## INTRODUÇÃO AO C

O C é, provavelmente, a linguagem de programação mais amplamente conhecida. Ela é usada como linguagem de referência para os cursos de ciência da computação em todo mundo, sendo, possivelmente, a linguagem que as pessoas mais aprendem nas escolas, juntamente com Python e Java.

A linguagem C é uma linguagem de programação de alto nível que foi criada nos anos 1970 para desenvolvimento de sistemas operacionais. Ela é amplamente utilizada em sistemas embarcados.

**Em programação C, bibliotecas** são conjuntos de funções e declarações que facilitam o desenvolvimento de software. Elas permitem reutilizar tarefas específicas, como entrada/saída, manipulações de strings, operações matemáticas, entre outras.

chamaremos de "Hello, World!" ("Olá, mundo!" em inglês).

```
#include <stdio.h>
```

```
int main(void) {  
    printf("Hello, World!");  
}
```

Vamos descrever o código-fonte do programa: primeiro, importaremos a biblioteca stdio (o nome significa "biblioteca de entrada e saída padrão" - em inglês, *STanDard Input and Output*).

Essa biblioteca nos dá acesso às funções de entrada e saída.

O C é uma linguagem muito pequena em seu núcleo. Tudo o que não é parte do núcleo é fornecido por meio de bibliotecas. Algumas dessas bibliotecas foram criadas por programadores "normais" e disponibilizadas para que outros as utilizassem. Algumas outras bibliotecas foram criadas dentro do compilador, como a stdio, entre outras.

A stdio é a biblioteca que fornece a função printf().

A função está envolvida em uma função main(). A função main() é o ponto de entrada de qualquer programa em C.

Uma função é uma rotina que recebe um ou mais argumentos e retorna um único valor.

No caso de `main()`, a função não recebe argumentos e retorna um número inteiro.

Identificamos isso usando a palavra-chave `void` como argumento e a palavra-chave `int` para o valor de retorno.

A função tem um corpo, que é envolvido por chaves. Dentro do corpo da função, temos todo o código de que a função necessita para realizar suas operações.

A função `printf()` está escrita de modo diferente, como é possível ver. Ela não tem um valor de retorno definido e passamos a ela uma string, envolvida por aspas duplas. Não especificamos o tipo de argumento.

Isso ocorre porque essa é uma invocação de função. Em algum lugar, dentro da biblioteca `stdio`, `printf` está definida assim:

```
int printf(const char *format, ...);
```

Você não precisa entender o que isso significa agora, mas, resumindo, essa é a definição.

Quando chamamos `printf("Hello, World!");`, a função será executada.

A função `main()` que definimos acima:

## Entrada e saída

### `scanf()`

`printf()` é usada como uma função de saída. Quero apresentar uma função de entrada agora, para podermos dizer que vimos tanto a entrada quanto a saída: `scanf()`.

Essa função é usada para obter de um usuário que estiver executando o programa um valor, a partir da linha de comando.

- `%c` para um caractere
- `%s` para um caractere
- `%f` para números de ponto flutuante
- `%p` para ponteiros

e muito mais.

```
#include <stdio.h>
```

```
int main(void) {  
    printf("Hello, World!");  
}
```

será executada pelo sistema operacional quando o programa for executado.

## Variáveis e tipos

**Variável:** é um local na memória do computador onde você pode armazenar um valor que pode ser alterado durante a execução do programa.

Ao criar uma variável em C, você tem de especificar o tipo de uma variável na declaração.

Neste exemplo, inicializamos a variável `age` com o tipo `int`:

```
int age;
```

O nome de uma variável pode conter letras maiúsculas e minúsculas, algarismo e o caractere de sublinha (*underscore*, em inglês), mas não podem começar por um algarismo. `AGE` e `Age10` são nomes válidos para variáveis, mas `1age` não é.

Você também pode iniciar uma variável no momento da declaração, especificando seu valor inicial:

```
int age = 37;
```

Ao declarar uma variável, você pode usá-la no código do programa. Você pode alterar seu valor a qualquer momento, usando o operador `=`, por exemplo, como em `age = 100`; (contanto que o novo valor seja do mesmo tipo).

Neste caso:

Em C, um comentário é uma sequência de caracteres que começa com `/*` e termina com `*/`. O compilador ignora os comentários, tratando-os como espaços em branco.

```
#include <stdio.h>
```

```
int main(void) {  
    int age = 0;  
    age = 37.2;
```

```

printf("%u", age);
}

1  #include <stdio.h>
2
3  int main(void) {
4  // Declara uma variável inteira chamada "idade"
5  int idade;
6
7  // Atribui o valor 25 à variável "idade"
8  idade = 25;
9
10 printf("Minha idade é: %d\n", idade);
11
12 return 0;
13

```

Os tipos de dados integrados do C são int, char, short, long, float, double e long double. Vejamos um pouco mais a respeito deles.

### Números inteiros

O C nos fornece os seguintes tipos para que possamos definir valores inteiros:

- char
- int
- short
- long

Na maior parte das vezes, você provavelmente usará um int para armazenar um número inteiro. Em alguns casos, no entanto, você pode querer escolher uma das outras três opções. O tipo char normalmente é usado para armazenar letras da tabela ASCII, mas pode ser usada para conter números inteiros pequenos, entre -128 e 127. Ele ocupa menos de 1 byte. int ocupa pelo menos 2 bytes. short ocupa pelo menos 2 bytes. long ocupa pelo menos 4 bytes.

Como você pode ver, não temos a garantia dos mesmos valores para ambientes diferentes. Temos apenas uma indicação. O problema é que os números exatos que podem ser armazenados em cada tipo de dados dependem da implementação e da arquitetura.

O que temos garantido é que short não é maior do que int e que long não é menor do que int. O padrão das especificações ANSI C determina os valores mínimos para cada tipo. Graças a isso, podemos ao menos saber qual é o valor mínimo que podemos esperar ter à nossa disposição.

Se você estiver programando em C em um Arduino, placas de circuito diferentes terão limites diferentes.

Em uma placa Arduino Uno, int armazena um valor de 2 bytes, indo de -32,768 a 32,767. Em uma placa Arduino MKR 1010, int armazena um valor de 4 bytes, indo de -2,147,483,648 a 2,147,483,647, o que traz uma grande diferença. Em todas as placas Arduino, short armazena um valor de 2 bytes, indo de -32,768 a 32,767. long armazena 4 bytes, indo de -2,147,483,648 a 2,147,483,647.

### Números inteiros sem sinal

Para todos os tipos de dados acima, podemos anexar antes deles a expressão unsigned (em português, sem sinal) para iniciar o intervalo em 0, em vez de em um número negativo. Isso pode fazer sentido em diversos casos.

- unsigned char irá de 0 a pelo menos 255
- unsigned int irá de 0 a pelo menos 65,535
- unsigned short irá de 0 a pelo menos 65,535
- unsigned long irá de 0 a pelo menos 4,294,967,295

### O problema com o overflow

Em função de todos esses limites, pode surgir uma pergunta: como podemos nos certificar de que nossos números não excedem o limite? O que acontece se excedermos o limite?

Se você tem um número unsigned int de 255 e somar mais um a ele, terá um retorno de 256, conforme o esperado. Se tiver um número unsigned char de 255 e somar mais um a ele, terá um retorno de 0. Ele retornará ao valor inicial possível.

Se você tiver um número unsigned char de 255 e adicionar 10 a ele, chegará ao número 9:

```
#include <stdio.h>
```

```
int main(void) {
unsigned char j = 255;
j = j + 10;
printf("%u", j); /* 9 */
}
```

Se você não tiver um valor com sinal, o comportamento é indefinido. Ele, basicamente, fornecerá um número enorme e que pode variar, como neste caso:

```
#include <stdio.h>

int main(void) {
char j = 127;
j = j + 10;
printf("%u", j); /* 4294967177 */
}
```

Em outras palavras, o C não protege você de passar dos limites de um tipo. Você é quem deve se preocupar com essa questão.

### Avisos ao declarar o tipo errado

Ao declarar a variável e inicializá-la com um valor do tipo incorreto, o compilador gcc (o que você provavelmente usará) deverá dar um aviso semelhante a este:

```
#include <stdio.h>
```

```
int main(void) {
char j = 1000;
}
```

```
hello.c:4:11: warning: implicit conversion
```

```
from 'int' to
```

```
'char' changes value from 1000 to -24
```

```
[-Wconstant-conversion]
```

```
char j = 1000;
```

```
1 warning generated.
```

Ele também avisará você no caso de atribuições diretas:

```
#include <stdio.h>
```

```
int main(void) {  
    char j;  
    j = 1000;  
}
```

Ele não fará isso, porém, se você aumentar o número usando, por exemplo, +=:

```
#include <stdio.h>
```

```
int main(void) {  
    char j = 0;  
    j += 1000;  
}
```

### Números de ponto flutuante

Os tipos de ponto flutuante (float, double e long double) podem representar um conjunto muito maior de valores do que os inteiros, podendo, também, representar frações, algo que os números inteiros não conseguem fazer.

Ao usar números de ponto flutuante, representamos números como decimais na potência de 10.

Você poderá ver números de ponto flutuante escritos assim:

- 1.29e-3
- -2.3e+5

além de outros escritos de maneira estranha.

Os tipos:

- float
- double
- long double

são usados para representar números com pontos decimais (tipos de ponto flutuante). Todos podem representar números positivos e negativos.

Os requisitos mínimos para qualquer implementação em C são de que float possa representar um intervalo entre  $10^{-37}$  e  $10^{+37}$ , sendo tipicamente implementados usando 32 bits. double pode representar um conjunto mais de números. long double pode ter ainda mais números.

Os valores exatos, assim como ocorre com os números inteiros, dependem da implementação.

Em um Mac moderno, um float é representado por 32 bits, tendo uma precisão de 24 bits significativos. 8 bits são usados para codificar o expoente.

Um número double é representado por 64 bits, com uma precisão de 53 bits significativos, sendo usados 11 bits para codificar o expoente.

O tipo long double é representado por 80 bits, tem uma precisão de 64 bits significativos, sendo usados 15 bits para codificar o expoente.

Em nosso computador especificamente, como podemos determinar o tamanho dos tipos?

Você pode escrever um programa que faça isso por você:

```
#include <stdio.h>

int main(void) {
    printf("Tamanho de char: %lu bytes\n", sizeof(char));
    printf("Tamanho de int: %lu bytes\n", sizeof(int));
    printf("Tamanho de short: %lu bytes\n", sizeof(short));
    printf("Tamanho de long: %lu bytes\n", sizeof(long));
    printf("Tamanho de float: %lu bytes\n", sizeof(float));
    printf("Tamanho de double: %lu bytes\n",
        sizeof(double));
    printf("Tamanho de long double: %lu bytes\n",
        sizeof(long double));
}
```

Em meu sistema, um Mac moderno, veremos:



Tamanho de char: 1 bytes

Tamanho de int: 4 bytes

Tamanho de short: 2 bytes

Tamanho de long: 8 bytes

Tamanho de float: 4 bytes

Tamanho de double: 8 bytes

Tamanho de long double: 16 bytes

## Constantes

Uma constante é declarada de modo semelhante às variáveis, exceto pelo fato de ter a palavra-chave `const` adicionada antes da declaração. Além disso, você sempre precisará adicionar na declaração um valor específico.

Por exemplo:

```
const int age = 37;
```

Esse código é perfeitamente válido em C, mas é comum declarar as constantes em letra maiúscula, assim:

```
const int AGE = 37;
```

É apenas uma convenção, mas pode ajudar muito ao ler ou escrever um programa em C, já que aumenta a legibilidade. Um nome em letras maiúsculas significa que é uma constante, enquanto, em letras minúsculas, temos uma variável.

Um nome de constante segue as mesmas regras dos nomes de variáveis: podem conter letras maiúsculas e minúsculas, algarismos e o caractere de sublinha, mas não pode começar com um algarismo. `AGE` e `Age10` são nomes de variável válidos, enquanto `1AGE` não é.

Outra maneira de definir constantes é usando essa sintaxe:

```
#define AGE 37
```

Neste caso, não é necessário adicionar um tipo, também não sendo preciso adicionar um sinal de igual `=`. Você deve omitir o ponto e vírgula do final.

O compilador do C fará a inferência do tipo a partir do valor especificado, em tempo de compilação.

## Operadores

O C nos oferece uma grande variedade de operadores que podemos usar para fazer operações com os dados.

Especificamente, podemos identificar alguns grupos de operadores:

- operadores aritméticos
- operadores de comparação
- operadores lógicos
- operadores de atribuição compostos
- operadores bitwise
- operadores de ponteiro
- operadores de estrutura
- operadores diversos

Nesta seção, detalharei todos eles usando 2 variáveis imaginárias, a e b, como exemplo.

Deixarei os operadores bitwise, de estrutura e de ponteiro fora dessa lista para manter o guia simples

### Operadores aritméticos

Neste grupo mais amplo, separarei os operadores binários e os unários.

Os operadores binários funcionam usando dois operandos:

Expressões aritméticas em C são combinações de operadores aritméticos e operandos numéricos. Os operadores aritméticos básicos são +, -, \*, / e %.

Operadores aritméticos + Soma, - Subtração, \* Multiplicação, / Divisão inteira, % Resto da divisão inteira.

| OPERADOR | NOME | EXEMPLO |
|----------|------|---------|
|----------|------|---------|

| OPERADOR | NOME          | EXEMPLO |
|----------|---------------|---------|
| =        | Atribuição    | a = b   |
| +        | Adição        | a + b   |
| -        | Subtração     | a - b   |
| *        | Multiplicação | a * b   |
| /        | Divisão       | a / b   |
| %        | Módulo        | a % b   |

Operadores unários recebem apenas um operando:

| OPERADOR | NOME                | EXEMPLO       |
|----------|---------------------|---------------|
| +        | Unário de soma      | +a            |
| -        | Unário de subtração | -a            |
| 1. ++    | 1. Incremento       | 2. a++ or ++a |
| 3. --    | 4. Decremento       | 5. a-- or --a |

A diferença entre a++ e ++a é que a++ incrementa a variável a após seu uso. ++a incrementa a variável a antes de usá-la.

Por exemplo:

```
int a = 2;
```

```
int b;
```

```
b = a++ /* b é 2, a é 3 */
```

```
b = ++a /* b é 4, a é 4 */
```

Isso também se aplica aos operadores de decremento.

### Operadores de comparação

| OPERADOR | NOME | EXEMPLO |
|----------|------|---------|
|----------|------|---------|

| OPERADOR | NOME                  | EXEMPLO |
|----------|-----------------------|---------|
| ==       | Operador de igual     | a == b  |
| !=       | Operador de diferente | a != b  |
| >        | Maior que             | a > b   |
| <        | Menor que             | a < b   |
| >=       | Maior que ou igual a  | a >= b  |
| <=       | Menor que ou igual a  | a <= b  |

### Operadores lógicos

- ! NOT (em português, "não" - exemplo: !a)
- && AND (em português, "e" - exemplo: a && b)
- || OR (em português, "ou" - exemplo: a || b)

Esses operadores são ótimos ao se trabalhar com valores booleanos.

### Operadores de atribuição compostos

Esses operadores são úteis para realizar uma atribuição e, ao mesmo tempo, realizar uma operação aritmética:

| OPERADOR | NOME                       | EXEMPLO |
|----------|----------------------------|---------|
| +=       | Atribuição e adição        | a += b  |
| -=       | Atribuição e subtração     | a -= b  |
| *=       | Atribuição e multiplicação | a *= b  |
| /=       | Atribuição e divisão       | a /= b  |
| %=       | Atribuição e módulo        | a %= b  |

### O operador ternário

O operador ternário é o único operador em C que funciona com 3 operandos, sendo um modo abreviado de expressar condicionais.

Ele tem essa aparência:

`<condição> ? <expressão> : <expressão>`

Exemplo:

`a ? b : c`

Se `a` for avaliado como `true`, a instrução `b` será executada. Caso contrário, é executada a instrução `c`.

O operador ternário, em termos de sua funcionalidade, é equivalente ao condicional `if/else`, exceto pelo fato de ser menor e de podermos colocar nele toda uma expressão em uma única linha.

### **sizeof**

O operador `sizeof` retorna o tamanho do operando que você passar. É possível passar uma variável ou, até mesmo, um tipo.

Exemplo de uso:

```
#include <stdio.h>
```

```
int main(void) {  
    int age = 37;  
    printf("%ld\n", sizeof(age));  
    printf("%ld", sizeof(int));  
}
```

### **Precedência dos operadores**

Com todos esses operadores (e mais, já que não tratamos de todos neste artigo, incluindo os operadores bitwise, de estrutura e de ponteiro), devemos prestar atenção a quando eles estiverem juntos em uma única expressão.

Vamos supor que temos esta operação:

```
int a = 2;
```

```
int b = 4;
```

```
int c = b + a * a / b - a;
```

Qual é o valor de c? A adição é executada antes da multiplicação e da divisão?

Existe um conjunto de regras que nos ajuda a resolver esse enigma.

Na ordem de menor precedência para a maior, temos:

- o operador de atribuição =
- os operadores **binários** + e -
- os operadores \* e /
- os operadores + e - unários

Os operadores também têm uma regra de associação, que é sempre da esquerda para a direita, exceto para os operadores unários e de atribuição.

Em:

```
int c = b + a * a / b - a;
```

Primeiro, executamos  $a * a / b$ , que, devido ao fato de ser da esquerda para a direita, separamos em  $a * a$  e o resultado  $/ b$ :  $2 * 2 = 4$ ,  $4 / 4 = 1$ .

Depois, realizamos a soma e a subtração:  $4 + 1 - 2$ . O valor de c é 3.

Em todos os casos, no entanto, eu gostaria de reforçar que é possível usar os parênteses de modo a tornar expressões semelhantes mais fáceis de ler e de compreender.

Os parênteses têm maior prioridade sobre todo o resto.

A expressão do exemplo acima pode ser reescrita assim:

```
int c = b + ((a * a) / b) - a;
```

e, desse modo, não temos que nos preocupar muito com ela.

## Condicionais

Todas as linguagens de programação fornecem aos programadores a capacidade de realizar escolhas.

**As estruturas de decisão em C são if, if-else e switch-case. Elas controlam o fluxo de execução do programa, permitindo que ele siga caminhos diferentes dependendo de condições.** Estruturas de decisão

O comando switch case em C

É uma forma de reduzir a complexidade de vários if ... else encadeados. É muito utilizado, principalmente para uso em estruturas de menu.

Estruturas de decisão encadeadas – if – else – if – else

Chamamos de estruturas de decisão encadeadas, quando uma estrutura de decisão está localizada dentro do lado falso da outra. Este tipo de estrutura também é conhecida como seleção “aninhada” ou seleção “encaixada”.

Estrutura de decisão if em Linguagem C

A estrutura de decisão if em Linguagem C Uma estrutura de decisão examina uma ou mais condições e decide quais instruções serão executadas dependendo se a condição foi ou não foi.

As estruturas de controle em C são comandos que determinam a ordem em que as instruções de um programa são executadas. Elas permitem que o programa execute diferentes ações de acordo com determinadas condições A estrutura sequencial em C é a sequência de instruções que compõem um programa, desde o início até o fim.

Estrutura básica de um programa em C

- Inclusão da biblioteca stdio.h, que contém funções de entrada e saída
- Definição da função main, que é o ponto de partida do programa
- Instruções do programa, que devem terminar com ponto-e-vírgula (;)
- Retorno do valor 0 ao final da função main, indicando que o programa terminou com sucesso

Dicas para programar em C

- Para facilitar a leitura, é importante indentar o código
- O laço for é uma estrutura de repetição muito utilizada em C, especialmente quando se sabe quantas vezes a repetição deve ser executada

- Outros comandos de repetição em C são while e do-while  
Considerações sobre a estrutura de dados
- Uma estrutura de dados sequencial é uma lista ordenada de elementos, em que cada elemento tem um predecessor e um sucessor, exceto o primeiro e o último
- Um algoritmo sequencial é uma sequência de instruções que são executadas em um tempo finito

Em alguns casos, queremos realizar a ação X. Em outros, a ação Y.

Queremos verificar os dados e fazer escolhas com base no estado desses dados.

O C nos fornece duas maneiras de fazer isso.

A primeira é a instrução if, com seu auxiliar else, e a segunda é a instrução switch.

#### if

Em uma instrução if (se, em português), você pode conferir se uma condição é verdadeira e executar o bloco fornecido entre chaves:

```
int a = 1;
```

```
if (a == 1) {
    /* Realizar uma ação */
}
```

Você pode anexar um bloco else (senão, em português) para executar um bloco diferente se a condição não for verdadeira (em outras palavras, se ela for falsa):

```
int a = 1;
```

```
if (a == 2) {
    /* Realizar uma ação */
} else {
    /* Realizar outra ação, caso contrário */
}
```

**Condicional composto: a estrutura if-else**



A estrutura if-else permite executar um bloco de código se uma condição for verdadeira e outro bloco de código se a condição for falsa. A sintaxe é a seguinte:

```
if (condicao) {  
// bloco de codigo a ser executado se a condicao for verdadeira  
} else {  
// bloco de codigo a ser executado se a condicao for falsa  
}
```

Por exemplo:

```
int num = -3;  
if (num > 0) {  
    printf("O numero e positivo");  
} else {  
    printf("O numero e negativo");  
}
```

Vamos supor que temos uma variável idade e queremos imprimir uma mensagem de acordo com a faixa etária. Podemos usar a estrutura if – else if – else para isso:

```
int idade = 32;  
if (idade < 18) {  
    printf("Você é menor de idade");  
} else if (idade >= 18 && idade <= 64) {  
    printf("Você é adulto");  
} else {  
    printf("Você é idoso");  
}
```

Cuidado com uma fonte comum de erros - sempre use o operador de comparação == para fazer comparações, não o operador de atribuição =. Do contrário, a verificação do condicional if será sempre verdadeira, a menos que o argumento seja 0. Por exemplo, se você executar esse código:

```
int a = 0;
```

```
if (a = 0) {  
    /* O que estiver aqui nunca será chamado */  
}
```

Por que isso acontece? Porque a verificação do condicional buscará um resultado booleano (*true/false*, verdadeiro ou falso, o resultado de uma comparação). O número 0 sempre será igual ao valor *false*. Todo o resto será igual a *true*, incluindo números negativos.

Você também pode ter diversos blocos de *else* empilhando-os na forma de instruções *else if*:

```
int a = 1;  
  
if (a == 2) {  
    /* Realizar uma ação */  
} else if (a == 1) {  
    /* Realizar outra ação */  
} else {  
    /* Realizar uma terceira ação */  
}
```

### **switch**

Caso precise fazer muitos blocos *if/else/else if* para realizar uma verificação, talvez por precisar verificar o valor exato de uma variável, a instrução *switch* (algo como *chave* em português) pode ser bastante útil para você.

Você pode fornecer uma variável como condição e uma série de pontos de entrada *case* (*caso*, em português) para cada valor que você espera:

```
int a = 1;  
  
switch (a) {  
    case 0:  
        /* Realizar uma ação */  
        break;  
    case 1:  
        /* Realizar outra ação */  
        break;
```

case 2:

```
/* Realizar outra ação */  
break;  
}
```

Precisamos usar a palavra-chave `break` ao final de cada *case* para evitar que o próximo *case* seja executado depois de o anterior ter sido concluído. Esse efeito "cascata" pode ser útil de maneiras bastante criativas.

Você pode usar um *case* que valha para várias situações ao final, chamado de default:

```
int a = 1;  
switch (a) {  
case 0:  
/* Realizar uma ação */  
break;  
case 1:  
/* Realizar outra ação */  
break;  
case 2:  
/* Realizar outra ação */  
break;  
default:  
/* Para todos os outros casos */  
break;  
}
```

As estruturas de repetição em C são o `for`, o `while` e o `do-while`. Essas estruturas permitem executar um conjunto de instruções repetidamente, enquanto uma determinada condição for verdadeira.

## Laços

O C nos oferece três maneiras de realizar um laço: **laços for**, **laços while** e **laços do while**.

Todos eles permitem percorrer os elementos de um array, mas com algumas diferenças.

Vamos vê-los em detalhes.

### Laços for

O primeiro modo e, provavelmente, o mais comum de se realizar um laço é usando **laços for**.

Usando a palavra-chave `for`, podemos definir as *regras* para o laço desde o início e, então, fornecer o bloco que será executado repetidamente.

Aqui vemos um exemplo:

```
for (int i = 0; i <= 10; i++) {  
    /* as instruções a serem repetidas */  
}
```

O bloco `(int i = 0; i <= 10; i++)` contém 3 partes com os detalhes do laço:

- a condição inicial `(int i = 0)`
- o teste `(i <= 10)`
- o incremento `(i++)`

Definimos, primeiramente, a variável do laço. Neste caso, a chamamos de `i`. `i` é uma variável comum que será usada em laços, assim como `j` para laços aninhados (um laço dentro de outro laço). Ambas são usadas apenas por convenção.

A variável é inicializada com o valor de 0 (zero) e a primeira iteração é realizada. Logo, vem a parte do incremento `(i++)`, neste caso, que incrementa a variável em 1), e todo o ciclo é repetido até você receber o número 10.

Dentro do bloco principal do laço, podemos acessar a variável `i` para saber em que iteração estamos. Este programa deve imprimir 0 1 2 3 4 5 5 6 7 8 9 10:

```
if (int i = 0; i <= 10; i++) {  
    /* as instruções a serem repetidas */  
    printf("%u ", i);  
}
```

Laços podem iniciar em um número maior e diminuir de acordo, assim:

```
for (int i = 10; i > 0; i--) {  
    /* as instruções a serem repetidas */  
}
```

Você também pode incrementar a variável de laço de 2 em 2 ou de acordo com algum outro valor:

```
for (int i = 0; i < 1000; i = i + 30) {  
    /* as instruções a serem repetidas */  
}
```

```
}
```

### Laços while

Os **laços while** são mais simples de escrever do que os laços for, mas exige um pouco mais de trabalho de sua parte.

Em vez de definir todo o laço de início ao começar o laço, como é feito no laço for, usando while, apenas verificamos uma condição:

```
while (i < 10) {
```

```
}
```

Esta maneira de escrever leva em consideração que i já esteja definida e inicializada com um valor.

Esse laço, porém, será um **laço infinito**, a menos que você incremente a variável i em algum momento, dentro do laço. Um laço infinito é ruim, pois bloqueará o programa, não permitindo que mais nada aconteça.

É isso que você precisa para fazer um laço while "correto":

```
# include <stdio.h>
```

```
Int main() {
```

```
    int i = 0;
```

```
    while (i < 10) {
```

```
        /* realizar uma ação */
```

```
        i++;
```

```
        printf("%u", i);
```

```
    }
```

```
    Return 0;
```

```
}
```

Existe uma exceção a isso - e a veremos em um minuto. Antes, permitam-me apresentar o laço do while.

### Laços do while

Embora os laços `while` sejam ótimos, pode haver situações em que você precise fazer uma coisa específica: executar um bloco ao menos uma vez *sempre* e, *talvez*, repeti-lo.

Isso é feito usando as palavras-chave do `while`. De certa forma, ele é muito semelhante a um laço `while`, com uma única diferença:

```
int i = 0;
do {
    /* realizar uma ação */
    i++;
} while (i < 10);
```

O bloco que contém o comentário `/* realizar uma ação */` sempre é executado ao menos uma vez, não importando a verificação da condição abaixo.

Então, enquanto a variável `i` for inferior a 10, o bloco será repetido.

### **Saindo de um laço usando *break***

Em todos os laços da linguagem C, temos um modo de sair do laço a qualquer momento, imediatamente, não importando as condições definidas pelo laço.

Isso é feito usando a palavra-chave `break`.

Isso é útil em várias situações. Você pode querer verificar o valor de uma variável, por exemplo:

```
for (int i = 0; i <= 10; i++) {
    if (i == 4 && someVariable == 10) {
        break;
    }
}
```

Essa opção de sair de um laço é particularmente interessante para laços `while` (e do `while`), pois podemos criar laços aparentemente infinitos que se encerram quando uma condição ocorre. Você define isso dentro do bloco do laço:

```
int i = 0;
while (1) {
    /* realizar uma ação */

    i++;
```

```
if (i == 10) break;  
}
```

É muito comum termos esse tipo de laço em C.

## Arrays

Um array é uma variável que armazena diversos valores.

Todos os valores em um array, em C, precisam ter o **mesmo tipo**. Em outras palavras, teremos apenas arrays de valores int, arrays de valores double e assim por diante.

Você pode definir um array de valores int, assim:

```
int prices[5];
```

Você deve sempre especificar o tamanho do array. O C não fornece arrays dinâmicos nativamente (é preciso usar uma estrutura de dados, como uma lista vinculada, para que isso seja possível).

Você pode usar uma constante para definir o tamanho:

```
const int SIZE = 5;
```

```
int prices[SIZE];
```

Você pode inicializar um no momento da definição, assim:

```
int prices[5] = { 1, 2, 3, 4, 5 };
```

Além disso, você pode atribuir um valor após a definição, desta forma:

```
int prices[5];
```

```
prices[0] = 1;
```

```
prices[1] = 2;
```

```
prices[2] = 3;
```

```
prices[3] = 4;
```

```
prices[4] = 5;
```

Ou ainda, de um modo mais prático, usando um laço:

```
int prices[5];
```

```
for (int i = 0; i < 5; i++) {  
    prices[i] = i + 1;  
}
```

Também pode referenciar um item no array usando colchetes após o nome da variável array, adicionando um número inteiro para determinar o valor do índice, assim:

```
prices[0]; /* valor do item do array: 1 */  
prices[1]; /* valor do item do array: 2 */
```

Os índices do array iniciam em 0 (zero). Desse modo, um array com 5 itens, como o array prices acima, terá itens que vão de prices[0] até prices[4].

Algo interessante sobre os arrays em C é o fato de que todos os elementos são armazenados em sequência, um após o outro. Isso não é algo que normalmente ocorra com linguagens de programação de alto nível.

Outra questão que vale a pena saber é sobre o nome da variável do array, como prices, no exemplo acima. O nome é um **ponteiro** do primeiro elemento do array. Assim, ele pode ser usado como um ponteiro normal.

## Strings

Em C, as strings são um tipo especial de array: uma string é um array de valores char:

```
char name[7];
```

Apresentei o tipo char quando falamos em tipos, mas, resumidamente, ele é comumente usado para armazenar letras da tabela ASCII.

Uma string pode ser inicializada da mesma maneira como se inicializa um array normal:

```
char name[7] = { "F", "l", "a", "v", "i", "o" };
```

Ou ainda, de modo mais conveniente, como uma *string literal* (também chamada de *string constante*), uma sequência de caracteres envoltas em aspas duplas:

```
char name[7] = "Flavio";
```

Você pode imprimir uma string com printf() usando %s:

```
printf("%s", name);
```



Percebeu como "Flavio" tem 6 caracteres, mas eu defini um array de tamanho 7? Por quê? Isso ocorre porque o último caractere de uma string deve ser um valor 0, o finalizador da string e precisamos deixar um espaço para ele. É importante ter isso em mente, especialmente ao manipular strings.

## Ponteiros

Os ponteiros são uma das partes mais confusas/desafiadoras da linguagem C, na minha opinião. Em especial, se você está recém começando em programação, mas também se você vier de uma linguagem de programação de alto nível, como o Python ou o JavaScript.

Nesta seção, eu quero apresentá-los da maneira mais simples possível, mas sem imbecilizar a definição.

Um ponteiro é o endereço de um bloco de memória que contém uma variável.

Usando `int *address` na declaração, não estamos declarando uma variável inteira, mas um **ponteiro para um número inteiro**.

Podemos usar o operador de ponteiro `*` para obter o valor da variável para a qual um endereço está apontando:

```
int age = 37;
int *address = &age;
printf("%u", *address); /* 37 */
```

Dessa vez, estamos usando o operador de ponteiro novamente, mas, como ele não é uma declaração agora, isso significa "o valor da variável para a qual esse ponteiro está apontando".

Neste exemplo, declaramos uma variável `age` e usamos um ponteiro para inicializar o valor:

```
int age;
int *address = &age;
*address = 37;
printf("%u", *address);
```

Arrays são um exemplo. Quando você declara um array:

```
int prices[3] = { 5, 4, 3 };
```

A variável `prices` é, de fato, um ponteiro para o primeiro item do array. Você pode obter o valor do primeiro item usando essa função `printf()`, neste caso:

```
printf("%u", *prices); /* 5 */
```

O interessante aqui é que podemos obter o segundo item adicionando 1 ao ponteiro de `prices`:

```
printf("%u", *(prices + 1)); /* 4 */
```

E assim por diante, para todos os outros valores.

Também podemos fazer muitas operações de manipulação de strings interessantes, já que strings são arrays internamente.

Também temos muitas outras aplicações, incluindo passar a referência de um objeto ou de uma função para evitar de consumir mais recursos ao copiar esse objeto ou essa função.

## Funções

As funções são uma forma através da qual podemos estruturar nosso código em sub-rotinas as quais podemos:

1. dar um nome
2. chamar quando precisamos delas

Iniciando de seu primeiro programa, um "Hello, World!", você já faz uso de funções em C imediatamente:

```
#include <stdio.h>
```

```
int main(void) {  
    printf("Hello, World!");  
}
```

A função `main()` é muito importante, pois é o ponto de entrada para um programa em C.

Aqui temos outra função:

```
void fazerAlgo(int value) {  
    printf("%u", value);  
}
```

As funções têm quatro aspectos importantes:

1. elas têm um nome, então podemos invocá-las ("chamá-las") mais tarde
2. elas especificam um valor de retorno
3. elas podem ter argumentos
4. elas têm um corpo, envolvido por chaves

O corpo da função é o conjunto de instruções que são executadas sempre que invocamos uma função.

Se a função não tiver um valor de retorno, você pode usar a palavra-chave `void` antes do nome da função. Do contrário, especifique o tipo do valor de retorno da função (`int` para inteiros, `float` para valores de ponto flutuante, `const char *` para uma string etc.). Você não pode retornar mais de um valor a partir de uma função.

Uma função pode ter argumentos. Eles são opcionais. Se ela não os tiver, dentro dos parênteses, inserimos `void`, assim:

```
void fazerAlgo(void) {
```

Neste caso, quando invocamos a função, a chamamos sem nada entre os parênteses:

```
fazerAlgo();
```

### **Escopo das variáveis**

Ao definir uma variável em um programa em C, dependendo de onde você a declarar, ela terá um **escopo** diferente.

Isso significa que ela estará disponível em alguns lugares, mas não em outros.

A posição determina 2 tipos de variáveis:

- **variáveis globais**
- **variáveis locais**

Esta é a diferença: uma variável declarada dentro de uma função é uma variável local, assim:

```
int main(void) {  
    int idade = 37;  
}
```

Variáveis locais estão acessíveis apenas dentro da função. Quando a função termina, elas deixam de existir. Elas são removidas da memória (com algumas exceções).

Uma variável definida fora de uma função é uma variável global, como neste exemplo:

```
int idade = 37;
```

```
int main(void) {  
    /* ... */  
}
```

As variáveis globais estarão acessíveis em qualquer função do programa, estando disponíveis durante toda a execução do programa, até que ele se encerre.

Já mencionei aqui que as variáveis locais não estão mais disponíveis depois do fim da função.

A razão é o fato de as variáveis locais serem declaradas na **stack** (em português, pilha) por padrão, a menos que você as aloque explicitamente na **heap** usando ponteiros. Nesse caso, você precisará gerenciar a memória por conta própria.

### Variáveis estáticas

Dentro de uma função, você pode inicializar uma **variável estática** usando a palavra-chave `static`.

Eu disse "dentro de uma função", pois as variáveis globais são estáticas por padrão, não havendo a necessidade de adicionar a palavra-chave.

O que é uma variável estática? Uma variável estática é inicializada se nenhum valor inicial for especificado, e retendo o valor em todas as chamadas da função.

Considere a função a seguir:

```
int aumentarIdade() {  
    int idade = 0;
```

```
idade++;  
return idade;  
}
```

Se chamarmos `aumentarIdade()` uma vez, teremos 1 como valor de retorno. Se a chamarmos mais de uma vez, receberemos sempre 1 como retorno, pois `idade` é uma variável local e reinicializada como 0 em cada chamada de função.

Se alterarmos a função para:

```
int aumentarIdade() {  
    static int idade = 0;  
    idade++;  
    return idade;  
}
```

Agora, toda vez que chamarmos essa função, teremos um valor aumentado em um:

```
printf("%d\n", aumentarIdade());  
printf("%d\n", aumentarIdade());  
printf("%d\n", aumentarIdade());  
retornará
```

```
1  
2  
3
```

Aqui temos um programa simples que usa o tipo enumerado acima:

```
#include <stdio.h>  
  
int main(void) {  
    DIA_DA_SEMANA dia = segunda;  
    if (dia == segunda) {  
        printf("Hoje é segunda!");  
    } else {  
        printf("Hoje não é segunda");  
    }  
}
```

```
#include "minhaPasta/meuArquivo.h"
```

Vejamos um exemplo. O programa a seguir calcula os anos que se passaram desde um determinado ano:

```
#include <stdio.h>
```

```
int calcularIdade(int ano) {  
    const int ANO_ATUAL = 2022;  
    return ANO_ATUAL - ano;  
}
```

```
int main(void) {  
    printf("%u", calcularIdade(1983));  
}
```

Vamos supor que eu queira mover a função `calcularIdade` para um arquivo em separado.

Criamos, então, um arquivo `calcular_idade.c`:

```
int calcularIdade(int ano) {  
    const int ANO_ATUAL = 2022;  
    return ANO_ATUAL - ano;  
}
```